RIBBAI OPS FOUNDATION RECORD

RIBBAI OPS has born

A professional summary of the architecture, infrastructure foundation, validation work, and current readiness of the RIBBAI restaurant operations platform.

Document purpose

This document explains what has been completed so far in the RIBBAI OPS project. It is a handoff-quality overview for stakeholders, future engineers, and decision makers before business modules are introduced.

PROJECT	CURRENT PHASE	STATUS
RIBBAI OPS	**Foundation Validation**	**Build-ready foundation**

Generated from the current workspace documentation and Phase 1 completion report.

Executive Summary

RIBBAI OPS has been established as a serious, production-oriented foundation for a restaurant operations management platform. The project is not yet a feature application. Instead, it now contains the architectural structure, core technical decisions, environment validation, database schema, authentication foundation, storage integration foundation, logging, error handling, audit logging, repository and service layer patterns, testing setup, and deployment readiness documentation needed before building business workflows.

> **Important boundary:** no inventory features, shift management, attendance flows, reporting workflows, dashboards, CRUD screens, or business modules have been implemented yet. The work completed so far is infrastructure and architecture only.

What Exists Today

AREA	CURRENT STATUS
Application shell	Minimal Next.js App Router shell with root layout, landing page, loading, error, and not-found boundaries.
Architecture	Enterprise folder structure and architecture documentation are in place.
Database	Prisma schema, generated client support, baseline migration SQL, and no-op seed entrypoint exist.
Authentication	Auth.js foundation exists with Prisma adapter and route handler, ready for Phase 2 providers and access control.
Storage	Supabase client and storage upload helper are configured as infrastructure.
Quality gates	ESLint, Prettier, TypeScript strict mode, Vitest, Playwright discovery, Husky hook, and CI workflow are configured.

Architectural Foundation

The project has been shaped around long-term maintainability. The folder structure separates routing, shared UI, domain features, server concerns, infrastructure utilities, tests, documentation, scripts, Prisma, reports, PDF templates, analytics, and future AI work.

Core architectural principles

- Domain-driven structure with clear feature boundaries.
- Server-first Next.js architecture using App Router conventions.
- Strict TypeScript with path aliases for maintainable imports.
- Repository and service layer foundations for future domain implementation.
- Runtime environment validation through Zod.
- Auditability and error handling designed before business workflows are added.

Primary technical stack

LAYER	TECHNOLOGY
Frontend	Next.js 15 App Router, React 19, TypeScript
UI foundation	TailwindCSS, shadcn-compatible Radix primitives, Lucide-ready dependency stack
Database	PostgreSQL with Prisma ORM
Authentication	Auth.js with Prisma adapter foundation
Storage	Supabase Storage client foundation
Testing	Vitest for unit tests and Playwright for E2E smoke coverage

Infrastructure Work Completed

Build-ready Next.js shell

- Created `app/layout.tsx`, `app/page.tsx`, and `app/globals.css`.
- Added global loading, not-found, and error boundaries.
- Added a health endpoint at `app/api/health/route.ts`.
- Kept the application shell intentionally neutral and non-business-specific.

Environment and configuration

- Created `.env.example` with application, database, Auth.js, Supabase, logging, audit, and testing variables.
- Added `lib/env` with Zod validation and safe local defaults for build tooling.
- Updated TypeScript configuration with `baseUrl` and strict path alias compatibility.
- Replaced the legacy ESLint setup with an ESLint 9 flat config.

Database foundation

- Configured a Prisma singleton at `lib/db/client.ts`.
- Generated a baseline migration SQL file.
- Added a safe no-op seed entrypoint.
- Verified Prisma Client generation and schema validation with a documented database URL.

Auth, storage, logging, and errors

- Added Auth.js root export and API route handler.
- Configured the Prisma Auth.js adapter and session type augmentation.
- Added Supabase browser and service client factories plus a storage upload helper.
- Added structured logging with environment-controlled log levels.
- Added typed application errors and normalization helpers.

Audit and backend patterns

- Added a reusable audit logging utility against the existing `AuditLog` Prisma model.
- Added `BaseRepository` for database and actor context.
- Added `BaseService` for service context and normalized error handling.
- Added centralized permission primitives for future role-based access control.

Validation Results

The Phase 1 success criteria were executed and passed. The project can now be treated as a build-ready foundation.

npm install: Passed
Dependencies installed and lockfile generated.

Prisma generate: Passed
Prisma Client generated successfully.

Lint: Passed
ESLint completed with no remaining errors.

TypeScript: Passed
Strict type checking completed successfully.

Build: Passed
Next.js production build completed successfully.

Unit tests: Passed
Vitest foundation test passed.

Additional verification

- Prisma schema validates when `DATABASE_URL` is provided.
- Playwright discovery finds the foundation smoke spec across configured browser projects.
- No IDE linter diagnostics were reported for the edited foundation areas.

What Was Intentionally Not Built

The current state is deliberately infrastructure-only. The following areas remain future phases and were not implemented as product features:

- Authentication screens and provider-specific login flows.
- Dashboard screens or operational widgets.
- Employee management workflows.
- Shift planning and attendance workflows.
- Inventory and supplier management workflows.
- Weekly inventory, reports, PDF report generation, and analytics features.
- Checklist, incident, document center, and AI business workflows.

Known Remaining Items

- **NPM audit findings:** dependency vulnerabilities were reported by npm and should be reviewed before production deployment.
- **Husky Git warning:** Husky reported that `.git` could not be found because the workspace was not detected as a Git repository.
- **Vitest warning:** Vite prints a CJS Node API deprecation warning, but tests pass.
- **Auth Phase 2:** providers, credentials, route protection, and role loading are intentionally pending.
- **Database application:** migration SQL exists, but applying it requires a live PostgreSQL connection.

Recommended Next Milestone

> **Phase 2: Authentication**
> The next milestone should implement secure authentication and authorization on top of the validated foundation: providers, login/logout flows, route protection, role and permission loading, and authentication audit events.

RIBBAI OPS is now ready to move from foundation into controlled feature development.